

Apache Spark Data Cleaning & Transformation

Assignment No-5: Apache Spark Fundamentals and Data Processing using PySpark

Dataset: Sample Superstore Dataset

Author: Snehal A. Bhosale

College: Sanjivani College of Engineering, Kopargaon – 423603

E-mail: snehalbhosale1807@gmail.com

Technology: • Apache Spark (PySpark) • Python • Google Colab • Jupyter Notebook

Introduction

Apache Spark is a powerful distributed data processing framework designed for large-scale analytics and high-performance computation. Unlike the traditional Hadoop MapReduce framework, Spark performs computations in memory, significantly improving execution speed for iterative algorithms, machine learning workloads, and real-time data processing.

This assignment demonstrates the complete lifecycle of data processing using Apache Spark DataFrames. The workflow includes loading a dataset, validating its schema, cleaning inconsistent records, handling missing values, transforming columns, performing aggregations, and generating business insights. The project concludes with a reusable ETL pipeline that reflects industry-standard data engineering practices.

Objectives

- Understand Apache Spark architecture.
- Learn Spark DataFrame operations.
- Load and inspect datasets.
- Perform data cleaning.
- Handle duplicate and missing values.
- Apply filtering and transformations.

- Perform aggregation and grouping.
- Understand shuffle operations.
- Build an end-to-end ETL pipeline.
- Generate business insights from sales data.

✓ Step1: Install Pyspark

```
!pip install pyspark -q
```

✓ Step 2 — Import Libraries

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import *

from pyspark.sql.types import *

from pyspark.sql.window import Window

import pandas as pd
```

✓ Step 3 — Load Dataset

```
# Load CSV file correctly
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("quote", "'') \
    .option("escape", "'') \
    .csv("Sample - Superstore.csv")

print("Dataset Loaded Successfully!")
```

```
Dataset Loaded Successfully!
```

✓ Step 4: Print Schema

```
df.printSchema()
```

```
root
|-- Row ID: integer (nullable = true)
|-- Order ID: string (nullable = true)
|-- Order Date: string (nullable = true)
|-- Ship Date: string (nullable = true)
|-- Ship Mode: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Customer Name: string (nullable = true)
|-- Segment: string (nullable = true)
|-- Country: string (nullable = true)
|-- City: string (nullable = true)
|-- State: string (nullable = true)
|-- Postal Code: integer (nullable = true)
|-- Region: string (nullable = true)
|-- Product ID: string (nullable = true)
|-- Category: string (nullable = true)
|-- Sub-Category: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Sales: double (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Discount: double (nullable = true)
|-- Profit: double (nullable = true)
```

✓ Step 5: Column Selection using DataFrame API

```
df.select("Product Name", "Sales").show(10, truncate=False)
```

```
+-----+-----+
|Product Name                                |Sales  |
+-----+-----+
|Bush Somerset Collection Bookcase           |261.96 |
|Hon Deluxe Fabric Upholstered Stacking Chairs, Rounded Back |731.94 |
|Self-Adhesive Address Labels for Typewriters by Universal    |14.62  |
|Bretford CR4500 Series Slim Rectangular Table               |957.5775|
|Eldon Fold 'N Roll Cart System                     |22.368 |
|Eldon Expressions Wood and Plastic Desk Accessories, Cherry Wood|48.86  |
|Newell 322                                           |7.28   |
|Mitel 5320 IP Phone VoIP phone                   |907.152|
|DXL Angle-View Binders with Locking Rings by Samsill        |18.504 |
|Belkin F5C206VTEL 6 Outlet Surge                  |114.9  |
+-----+-----+
only showing top 10 rows
```

✓ Step 6: Display Dataset Preview

```
df.show(5)
```

```
+-----+-----+-----+-----+-----+-----+-----+
|Row ID|      Order ID|Order Date| Ship Date|      Ship Mode|Customer ID| Cus
+-----+-----+-----+-----+-----+-----+-----+
|      1|CA-2016-152156| 11/8/2016|11/11/2016|    Second Class|    CG-12520|    C
|      2|CA-2016-152156| 11/8/2016|11/11/2016|    Second Class|    CG-12520|    C
|      3|CA-2016-138688| 6/12/2016| 6/16/2016|    Second Class|    DV-13045|Darri
|      4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|    SO-20335| Sean
|      5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|    SO-20335| Sean
+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

✓ Step 7:Inspect DataFrame Schema

```
df.printSchema()
```

```
root
|-- Row ID: integer (nullable = true)
|-- Order ID: string (nullable = true)
|-- Order Date: string (nullable = true)
|-- Ship Date: string (nullable = true)
|-- Ship Mode: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Customer Name: string (nullable = true)
|-- Segment: string (nullable = true)
|-- Country: string (nullable = true)
|-- City: string (nullable = true)
|-- State: string (nullable = true)
|-- Postal Code: integer (nullable = true)
|-- Region: string (nullable = true)
|-- Product ID: string (nullable = true)
|-- Category: string (nullable = true)
|-- Sub-Category: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Sales: double (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Discount: double (nullable = true)
|-- Profit: double (nullable = true)
```

✓ Step 8: Count Total Rows and Columns

```
print("Total Rows :", df.count())  
print("Total Columns :", len(df.columns))
```

```
Total Rows : 9994  
Total Columns : 21
```

Observation:

The Sample Superstore dataset has been successfully loaded into a Spark DataFrame and validated by inspecting its structure, schema, and dimensions. The initial exploration confirms that the dataset is ready for further processing. These validation steps help identify potential data quality issues before analysis. The DataFrame will now be used for data cleaning, transformation, filtering, aggregation, and business insight generation using PySpark.

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer

Traditional MapReduce is heavily constrained by its disk-bound architecture, rigid two-stage execution, and lack of native support for iterative or real-time processing. Spark addresses these issues by leveraging in-memory caching, lazy evaluation via Directed Acyclic Graphs (DAG), and high-level abstractions, making it vastly faster and more versatile.

The key limitations of MapReduce that drive the preference for Spark include:

High Disk Latency: MapReduce forces disk writes after every step. Spark processes data directly in RAM.

Slow Machine Learning: Iterative algorithms must reload data from disk each cycle. Spark holds datasets in memory.

Batch-Only Design: MapReduce lacks native real-time streaming. Spark handles batch, streaming, and SQL natively.

Rigid Programming: Code is complex and strictly limited to map and reduce. Spark offers flexible APIs.

No Built-in Tools: MapReduce lacks native machine learning libraries. Spark includes MLlib and Spark SQL.

Q2. Explain how Spark uses In-Memory Computing to speed up

Explain how Spark uses in-memory computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer

Apache Spark uses in-memory computing, which means it stores intermediate data in RAM instead of writing it to disk after every operation. Since accessing data from memory is much faster than reading and writing to disk, Spark can process data more efficiently.

This approach is especially useful for iterative machine learning algorithms because the same dataset is processed multiple times during training. Spark keeps the required data in memory, reducing repeated disk access and significantly improving execution speed.

In contrast, traditional disk-based systems like MapReduce store intermediate results on disk after each processing stage. This increases input/output operations and makes repeated computations much slower.

Advantages of In-Memory Computing in Spark:

- Faster execution due to reduced disk I/O.
- Efficient processing of iterative machine learning algorithms.
- Better performance for data analysis and transformations.
- Supports real-time and interactive data processing.
- Reduces overall execution time for large datasets.

Q3. Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Answer

```
print(df.columns)
```

```
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID',
```

The given dataset does not contain the columns `user_id` and `transaction_date`.

Therefore, the equivalent columns Customer ID and Order Date are used to demonstrate the same operation.

The dropDuplicates() function removes duplicate records based on the specified columns while keeping the first occurrence of each unique combination.

```
# Remove duplicate rows based on Customer ID and Order Date

df_no_duplicates = df.dropDuplicates(["Customer ID", "Order Date"])

print("Original Record Count :", df.count())
print("Record Count After Removing Duplicates :", df_no_duplicates.count())

df_no_duplicates.show(5)
```

```
Original Record Count : 9994
Record Count After Removing Duplicates : 4992
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name
1300	CA-2015-121391	10/4/2015	10/7/2015	First Class	AA-10315	Alejandro
5199	CA-2016-103982	3/3/2016	3/8/2016	Standard Class	AA-10315	Alejandro
2230	CA-2014-128055	3/31/2014	4/5/2014	Standard Class	AA-10315	Alejandro
1160	CA-2017-147039	6/29/2017	7/4/2017	Standard Class	AA-10315	Alejandro
7469	CA-2014-138100	9/15/2014	9/20/2014	Standard Class	AA-10315	Alejandro

only showing top 5 rows

✓ Insight

Duplicate records based on Customer ID and Order Date were removed using the dropDuplicates() function. This helps improve data quality by eliminating redundant entries before performing further analysis or transformations.

- ✓ **Q4. Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.**

Answer

The given dataset contains the columns Region, Category, and Sales, which are equivalent to the columns mentioned in the question.

In this solution, the data is first filtered to include only records where the Region is West. The filtered data is then grouped by Category, and the average Sales value is calculated using the avg() aggregation function.

```
from pyspark.sql.functions import col

# Convert numeric columns to proper data types

df = df.withColumn("Sales", col("Sales").cast("double")) \
        .withColumn("Quantity", col("Quantity").cast("int")) \
        .withColumn("Discount", col("Discount").cast("double"))
```

```
df.printSchema()
```

```
root
|-- Row ID: integer (nullable = true)
|-- Order ID: string (nullable = true)
|-- Order Date: string (nullable = true)
|-- Ship Date: string (nullable = true)
|-- Ship Mode: string (nullable = true)
|-- Customer ID: string (nullable = true)
|-- Customer Name: string (nullable = true)
|-- Segment: string (nullable = true)
|-- Country: string (nullable = true)
|-- City: string (nullable = true)
|-- State: string (nullable = true)
|-- Postal Code: integer (nullable = true)
|-- Region: string (nullable = true)
|-- Product ID: string (nullable = true)
|-- Category: string (nullable = true)
|-- Sub-Category: string (nullable = true)
|-- Product Name: string (nullable = true)
|-- Sales: double (nullable = true)
|-- Quantity: integer (nullable = true)
|-- Discount: double (nullable = true)
|-- Profit: double (nullable = true)
```

```
df.select("Sales").summary().show()
```

```
+-----+-----+
|summary|      Sales|
+-----+-----+
|  count|      9994|
|   mean|229.8580008304938|
| stddev|623.2451005086809|
|    min|         0.444|
|   25%|        17.248|
|   50%|        54.384|
|   75%|       209.94|
|    max|    22638.48|
```

```
+-----+-----+-----+
```

✓ Data Cleaning and Schema Correction

Before performing analytical operations, the numerical columns were converted to their appropriate data types. This ensures that aggregation functions such as average, sum, minimum, and maximum work correctly.

```
from pyspark.sql.functions import avg, col

avg_sales = (
    df.filter(col("Region") == "West")
      .groupBy("Category")
      .agg(avg("Sales").alias("Average Sales"))
)

avg_sales.show()
```

```
+-----+-----+-----+
|      Category|  Average Sales|
+-----+-----+-----+
|Office Supplies|116.422376910912|
|      Furniture|357.302324611033|
|      Technology|420.687532554257|
+-----+-----+-----+
```

Q5. What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

✓ Answer

`.na.drop()` removes rows (or columns) that contain null values. It is useful when incomplete records are not required for further analysis.

`.na.fill()` replaces null values with a specified value instead of removing the records. This helps preserve the dataset while handling missing values.

In the following example, all null values in the status column are replaced with the string "Unknown".

```

from pyspark.sql import Row

data = [
    Row(id=1, status="Active"),
    Row(id=2, status=None),
    Row(id=3, status="Inactive"),
    Row(id=4, status=None)
]

status_df = spark.createDataFrame(data)

print("Original DataFrame")
status_df.show()

print("After replacing null values")
status_df.na.fill({"status": "Unknown"}).show()

```

Original DataFrame

```

+---+-----+
| id|  status|
+---+-----+
|  1|  Active|
|  2|    NULL|
|  3|Inactive|
|  4|    NULL|
+---+-----+

```

After replacing null values

```

+---+-----+
| id|  status|
+---+-----+
|  1|  Active|
|  2| Unknown|
|  3|Inactive|
|  4| Unknown|
+---+-----+

```

Note: Since the provided Superstore dataset does not include a status column, a sample DataFrame was created to demonstrate how the `.na.fill()` function replaces null values with "Unknown" as required by the question.

- ✓ **Q6. Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.**

Answer

The dataset is grouped by the City column using the `groupBy()` function. The `count()` aggregation function calculates the total number of records for each city. Finally, the `filter()` function is used to display only those cities where the record count is greater than 100.

```
from pyspark.sql.functions import count, col

city_count = (
    df.groupBy("City")
      .agg(count("*").alias("Total Records"))
      .filter(col("Total Records") > 100)
      .orderBy(col("Total Records").desc())
)

city_count.show()
```

```
+-----+-----+
|      City|Total Records|
+-----+-----+
|New York City|      915|
|  Los Angeles|      747|
|Philadelphia|      537|
|San Francisco|      510|
|    Seattle|      428|
|    Houston|      377|
|    Chicago|      314|
|    Columbus|      222|
|  San Diego|      170|
|Springfield|      163|
|    Dallas|      157|
|Jacksonville|      125|
|    Detroit|      115|
+-----+-----+
```

Insight

The result shows the cities with more than 100 records in the dataset. These cities represent the highest transaction volumes and can be considered major business locations for further sales and customer analysis.

Q7. How does the immutability of Spark DataFrames affect how

- ✓ **you perform "data cleaning" steps like dropping columns or renaming them?**

Answer

Spark DataFrames are immutable, which means their data cannot be modified after they are created. Any operation such as dropping a column, renaming a column, or filtering data does not change the original DataFrame. Instead, Spark creates a new DataFrame containing the requested changes.

This approach helps maintain data consistency, supports fault tolerance, and enables Spark to optimize query execution efficiently.

```
# Original DataFrame
df.show(5)

# Create a new DataFrame after dropping a column
new_df = df.drop("Postal Code")

print("Original DataFrame Columns:")
print(df.columns)

print("\nNew DataFrame Columns:")
print(new_df.columns)
```

```
+-----+-----+-----+-----+-----+-----+-----+
|Row ID|      Order ID|Order Date| Ship Date|      Ship Mode|Customer ID|  Cus
+-----+-----+-----+-----+-----+-----+-----+
|      1|CA-2016-152156| 11/8/2016|11/11/2016|    Second Class|    CG-12520|    C
|      2|CA-2016-152156| 11/8/2016|11/11/2016|    Second Class|    CG-12520|    C
|      3|CA-2016-138688| 6/12/2016| 6/16/2016|    Second Class|    DV-13045|Darri
|      4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|    SO-20335| Sean
|      5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|    SO-20335| Sean
+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
Original DataFrame Columns:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID',
New DataFrame Columns:
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID',
```

Insight

The original DataFrame remains unchanged after performing the drop() operation. A new DataFrame is created with the modified schema, demonstrating the immutable nature of Spark DataFrames.

- Q8. Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Answer

The provided Superstore dataset does not contain Age and Subscription columns. Therefore, a sample DataFrame has been created to demonstrate how Spark filters records based on multiple conditions.

The filter selects records where the Age is between 18 and 30 (inclusive) and the Subscription type is "Premium".

```
from pyspark.sql import Row
from pyspark.sql.functions import col

# Sample DataFrame
data = [
    Row(Name="Alice", Age=22, Subscription="Premium"),
    Row(Name="Bob", Age=17, Subscription="Premium"),
    Row(Name="Charlie", Age=28, Subscription="Basic"),
    Row(Name="David", Age=30, Subscription="Premium"),
    Row(Name="Eva", Age=25, Subscription="Premium")
]

sample_df = spark.createDataFrame(data)

print("Original DataFrame")
sample_df.show()

filtered_df = sample_df.filter(
    (col("Age").between(18, 30)) &
    (col("Subscription") == "Premium")
)

print("Filtered DataFrame")
filtered_df.show()
```

```
Original DataFrame
+-----+---+-----+
|  Name|Age|Subscription|
+-----+---+-----+
|  Alice| 22|    Premium|
|   Bob| 17|    Premium|
|Charlie| 28|     Basic|
| David| 30|    Premium|
```

```

|      Eva| 25|      Premium|
+-----+---+-----+

Filtered DataFrame
+-----+---+-----+
| Name|Age|Subscription|
+-----+---+-----+
| Alice| 22|      Premium|
| David| 30|      Premium|
|   Eva| 25|      Premium|
+-----+---+-----+

```

Insight

The filtering operation combines multiple conditions using the AND (&) operator. Only records satisfying both conditions, Age between 18 and 30 and Premium subscription, are included in the final result.

- ✓ **Q9. When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?**

Answer

It is important to handle null values before performing aggregation functions such as `sum()` or `avg()` because missing values can affect the accuracy of the results.

Replacing or removing null values ensures that calculations are performed on valid data and prevents misleading business insights.

In Spark, null values can be handled using functions such as `.na.drop()` or `.na.fill()` depending on the data cleaning requirement.

```

from pyspark.sql import Row

# Sample DataFrame with null values
data = [
    Row(Product="Laptop", Price=50000),
    Row(Product="Mouse", Price=None),
    Row(Product="Keyboard", Price=1500),
    Row(Product="Monitor", Price=None)
]

price_df = spark.createDataFrame(data)

```

```
print("Original DataFrame")
price_df.show()

# Fill null values with 0
clean_df = price_df.na.fill({"Price": 0})

print("After Handling Null Values")
clean_df.show()
```

Original DataFrame

```
+-----+-----+
| Product|Price|
+-----+-----+
| Laptop|50000|
| Mouse| NULL|
| Keyboard| 1500|
| Monitor| NULL|
+-----+-----+
```

After Handling Null Values

```
+-----+-----+
| Product|Price|
+-----+-----+
| Laptop|50000|
| Mouse| 0|
| Keyboard| 1500|
| Monitor| 0|
+-----+-----+
```

Insight

Handling null values before performing aggregation ensures more reliable and meaningful analytical results. Depending on the business requirement, null values can either be removed or replaced with appropriate default values.

- ✓ **Q10. Write the code to revise a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.**

Answer

The given Superstore dataset does not contain a `raw_timestamp` column. Therefore, a small sample DataFrame was created to demonstrate how Spark converts a string timestamp into `TimestampType` and renames the column to `event_time`.

timestamp into timestamp type and renames the column to event_time.

```
from pyspark.sql import Row
from pyspark.sql.functions import col
from pyspark.sql.types import TimestampType

# Sample DataFrame
timestamp_data = [
    Row(raw_timestamp="2026-07-18 10:15:30"),
    Row(raw_timestamp="2026-07-19 14:45:10")
]

timestamp_df = spark.createDataFrame(timestamp_data)

print("Original DataFrame")
timestamp_df.show(truncate=False)

# Cast and rename
updated_df = (
    timestamp_df
    .withColumn("raw_timestamp", col("raw_timestamp").cast(TimestampType()))
    .withColumnRenamed("raw_timestamp", "event_time")
)

print("Updated DataFrame")
updated_df.show(truncate=False)

print("Updated Schema")
updated_df.printSchema()
```

Original DataFrame

```
+-----+
|raw_timestamp|
+-----+
|2026-07-18 10:15:30|
|2026-07-19 14:45:10|
+-----+
```

Updated DataFrame

```
+-----+
|event_time|
+-----+
|2026-07-18 10:15:30|
|2026-07-19 14:45:10|
+-----+
```

Updated Schema

```
root
 |-- event_time: timestamp (nullable = true)
```

Insight

Casting string values to TimestampType allows Spark to perform date and time-based operations efficiently. Renaming the column to event_time also improves readability and makes the dataset easier to understand during analysis.

Q11. Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer

A shuffle is the process in Spark where data is redistributed across different partitions so that records with the same key are brought together. This usually happens during operations such as groupBy(), join(), and reduceByKey().

Shuffle is considered a wide transformation because data is moved across multiple partitions and executors in the cluster. Unlike narrow transformations, where each partition depends on only one parent partition, wide transformations require data exchange over the network, making them more expensive in terms of execution time and resources.

```
from pyspark.sql.functions import sum

# Group data by Category and calculate total sales
category_sales = (
    df.groupBy("Category")
      .agg(sum("Sales").alias("Total Sales"))
)

category_sales.show()
```

```
+-----+-----+
|      Category|      Total Sales|
+-----+-----+
|Office Supplies|719047.032000029|
|      Furniture|741999.7952999998|
|      Technology|836154.0329999966|
+-----+-----+
```

Insight

The groupBy() operation requires Spark to redistribute records with the same

category into the same partition before calculating the total sales. This data movement is known as shuffle, making `groupBy()` a wide transformation because it involves communication between multiple partitions.

Q12. Write a code snippet that identifies and removes rows

- ✓ **where the email column contains null values OR the username is an empty string.**

Answer

The provided Superstore dataset does not contain email and username columns. Therefore, a sample DataFrame has been created to demonstrate how Spark identifies and removes rows where the email is null or the username is an empty string.

This helps improve data quality by removing incomplete or invalid records before further analysis.

```
from pyspark.sql import Row
from pyspark.sql.functions import col

# Sample DataFrame
data = [
    Row(username="Alice", email="alice@gmail.com"),
    Row(username="", email="bob@gmail.com"),
    Row(username="Charlie", email=None),
    Row(username="David", email="david@gmail.com"),
    Row(username="", email=None)
]

user_df = spark.createDataFrame(data)

print("Original DataFrame")
user_df.show()

# Remove rows with null email OR empty username
clean_df = user_df.filter(
    (col("email").isNotNull()) &
    (col("username") != "")
)

print("Cleaned DataFrame")
clean_df.show()
```

Original DataFrame

```
+-----+-----+
|username|      email|
+-----+-----+
|  Alice |alice@gmail.com|
|    Bob | bob@gmail.com|
| Charlie|      NULL|
|  David |david@gmail.com|
|       |      NULL|
+-----+-----+
```

Cleaned DataFrame

```
+-----+-----+
|username|      email|
+-----+-----+
|  Alice |alice@gmail.com|
|  David |david@gmail.com|
+-----+-----+
```

Insight

Filtering invalid records before analysis improves data quality and ensures that only complete and meaningful records are processed. This reduces errors and increases the reliability of analytical results.

Q13. How do you use the .agg() function to calculate multiple

- ✓ **statistics at once, such as the min, max, and mean of the price column?**

Answer

The given Superstore dataset contains a Sales column instead of a Price column. Therefore, the Sales column is used to demonstrate the .agg() function.

The .agg() function allows multiple aggregation operations to be performed in a single query. In this example, it is used to calculate the minimum, maximum, and average sales values.

```
from pyspark.sql.functions import min, max, avg

sales_stats = df.agg(
    min("Sales").alias("Minimum Sales"),
    max("Sales").alias("Maximum Sales"),
    avg("Sales").alias("Average Sales")
)
```

```

/

sales_stats.show()

```

```

+-----+-----+-----+
|Minimum Sales|Maximum Sales|    Average Sales|
+-----+-----+-----+
|          0.444|        22638.48|229.8580008304938|
+-----+-----+-----+

```

Insight

Using the `.agg()` function makes it possible to calculate multiple statistical measures in a single operation. This improves efficiency and reduces the need for writing separate aggregation queries for each metric.

- ✓ **Q14. In the context of cleaning a dataset, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?**

Answer

The `inferSchema=true` option allows Spark to automatically detect the data type of each column while reading a dataset. However, if the source data contains inconsistent or messy date formats, Spark may infer an incorrect data type or treat the entire column as a string.

This can lead to errors during data processing, incorrect sorting or filtering of dates, and failures in date-based calculations. For reliable data processing, it is often recommended to define the schema explicitly or clean the data before performing analysis.

```

from pyspark.sql import Row

# Sample DataFrame with inconsistent date formats
data = [
    Row(raw_date="2026-07-18"),
    Row(raw_date="18/07/2026"),
    Row(raw_date="July 18, 2026"),
    Row(raw_date="2026/07/18")
]

```

```
date_df = spark.createDataFrame(data)

print("Sample Data")
date_df.show(truncate=False)

print("Inferred Schema")
date_df.printSchema()
```

```
Sample Data
+-----+
|raw_date|
+-----+
|2026-07-18|
|18/07/2026|
|July 18, 2026|
|2026/07/18|
+-----+

Inferred Schema
root
 |-- raw_date: string (nullable = true)
```

Insight

When date values are stored in multiple formats, Spark cannot reliably determine a single date type and often treats the column as a string. Defining the schema manually or standardizing date formats before loading the data helps avoid incorrect data types and improves data quality.

✓ Q15. Write a final processing pipeline that:

```
Filters out duplicates.
Fills null prices with 0.
Groups by store_id to calculate total revenue.
```

Answer

The provided Superstore dataset does not contain `store_id` or `price` columns. Therefore, Customer ID is used as the grouping column, while Sales is used as the revenue column.

The pipeline performs the following operations:

```
Removes duplicate records.
```

Replaces null values in the Sales column with 0.
 Groups records by Customer ID.
 Calculates the total revenue for each customer using the sum() aggregation function.

```
from pyspark.sql.functions import sum

# Step 1: Remove duplicate records
clean_df = df.dropDuplicates()

# Step 2: Fill null values in Sales with 0
clean_df = clean_df.na.fill({"Sales": 0})

# Step 3: Group by Customer ID and calculate total revenue
revenue_df = (
    clean_df.groupBy("Customer ID")
              .agg(sum("Sales").alias("Total Revenue"))
              .orderBy("Total Revenue", ascending=False)
)

revenue_df.show(10)
```

```
+-----+-----+
|Customer ID|    Total Revenue|
+-----+-----+
|  SM-20320|         25043.05|
|  TC-20980|         19052.218|
|  RB-19360|         15117.339|
|  TA-21385|         14595.62|
|  AB-10105|14473.570999999998|
|  KL-16645|14175.228999999998|
|  SC-20095|14142.333999999999|
|  HL-15040|12873.297999999999|
|  SE-20110|12209.438000000002|
|  CC-12370|         12129.072|
+-----+-----+
only showing top 10 rows
```

Insight

This pipeline demonstrates a complete Spark DataFrame workflow by combining data cleaning, null value handling, grouping, and aggregation into a single process. Such pipelines are commonly used in real-world data engineering projects to prepare raw data for analysis and reporting.

✓ Final Data Processing Pipeline

This section integrates all the previous data processing steps into a single, reusable ETL pipeline. The pipeline reads the raw dataset, validates its structure, performs data cleaning, applies necessary transformations, computes key business metrics, and exports the processed results. This modular workflow demonstrates a production-oriented approach to building scalable Spark data processing applications.

```
# =====  
# Final Data Processing Pipeline  
# =====  
  
print("Starting Data Processing Pipeline...")  
  
# Use the existing DataFrame  
pipeline_df = df  
  
# Remove Duplicates  
pipeline_df = pipeline_df.dropDuplicates()  
  
# Fill Missing Values  
pipeline_df = pipeline_df.fillna({  
    "sales": 0,  
    "quantity": 0,  
    "discount": 0,  
    "profit": 0  
})  
  
# Standardize Column Names  
for c in pipeline_df.columns:  
    pipeline_df = pipeline_df.withColumnRenamed(  
        c,  
        c.lower().replace(" ", "_").replace("-", "_")  
    )  
  
# Business Summary  
business_summary = (  
    pipeline_df.groupBy("region")  
        .agg(  
            sum("sales").alias("total_sales"),  
            sum("profit").alias("total_profit")  
        )  
    .orderBy(desc("total_sales"))  
)  
  
business_summary.show()  
  
print("Pipeline Executed Successfully!")
```



```
Starting Data Processing Pipeline...
```

```
+-----+-----+-----+
| region|    total_sales|    total_profit|
+-----+-----+-----+
|   West|725457.8245000001|    108418.4489|
|   East|678781.2400000005|     91522.78|
|Central|501239.8907999995|    39706.3625|
|   South|    391721.905|46749.43030000002|
+-----+-----+-----+
```

```
Pipeline Executed Successfully!
```

✓ Results Summary

This section presents the final outcomes of the data processing workflow. It summarizes the key dataset statistics, business metrics, and overall processing results obtained after completing data cleaning, transformation, and aggregation. These results provide a concise overview of the effectiveness of the implemented Spark pipeline.

```
print(df.columns)
```

```
['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID',
```

```
print("="*60)
print("PROJECT RESULTS SUMMARY")
print("="*60)

print("Total Records :", df.count())
print("Total Columns :", len(df.columns))

print("Unique Customers :", df.select("Customer ID").distinct().count())
print("Unique Products :", df.select("Product ID").distinct().count())
print("Regions :", df.select("Region").distinct().count())
print("Categories :", df.select("Category").distinct().count())

print("="*60)
```

```
=====
PROJECT RESULTS SUMMARY
=====
Total Records : 9994
Total Columns : 21
Unique Customers : 793
Unique Products : 1862
Regions : 4
Categories : 3
=====
```

Learning Outcomes

During this assignment, the following concepts and practical skills were learned:

- Understood Apache Spark architecture and its advantages over Hadoop MapReduce.
- Learned how Spark DataFrames enable efficient large-scale data processing.
- Loaded and explored structured datasets using PySpark.
- Performed data cleaning by handling missing values and removing duplicate records.
- Applied filtering, sorting, and transformation operations on DataFrames.
- Used aggregation functions such as `sum()`, `avg()`, `min()`, `max()`, and `count()`.
- Performed grouping operations using `groupBy()` and analyzed shuffle behavior.
- Understood the concept of DataFrame immutability and lazy evaluation.
- Built a complete end-to-end ETL pipeline using Spark DataFrames.
- Generated business insights from the Sample Superstore dataset through sales and profit analysis.
- Improved understanding of scalable data engineering workflows and Spark best practices.

✓ Conclusion

This assignment provided practical experience with Apache Spark and PySpark DataFrames for large-scale data processing. The workflow included data loading, validation, cleaning, transformation, aggregation, and business analysis using the Sample Superstore dataset.

The assignment also demonstrated Spark's advantages over traditional MapReduce through in-memory processing, lazy evaluation, and efficient distributed execution. Finally, a robust ETL pipeline was developed to automate the complete data processing workflow, highlighting Spark's capability to process structured data efficiently and generate meaningful business insights.

Overall, this project strengthened practical knowledge of Spark fundamentals, DataFrame operations, data engineering workflows, and scalable data processing techniques.

Start coding or generate with AI.